

Department of Computer Science

University of Engineering and Technology (UET), Lahore

Data Structures and Algorithms Lab

Lab Manual

Lab No. 05

Title: Queue Implementation and Applications

1. Lab Objectives

After completing this lab, students will be able to:

- understand the concept of **FIFO (First In First Out)**
- implement queue using **array**
- implement queue using **linked list**
- understand and implement **circular queue**
- analyze **time and space complexity** of queue operations
- identify and solve **queue overflow/underflow problems**
- understand optimization scenarios (circular queue, dynamic memory)
- apply queue in solving real-world problems
- implement **queue-based algorithms**

2. Introduction

A **queue** is a linear data structure that follows the **FIFO principle**:

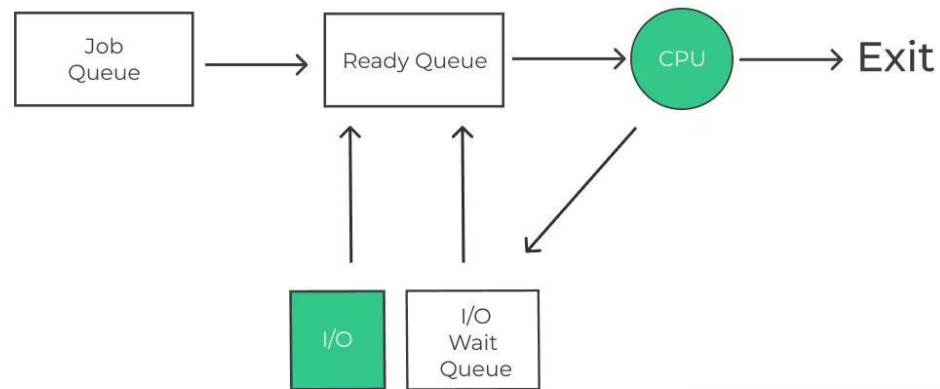
First In, First Out

This means the element inserted first is removed first.

Real-Life Examples



Process Scheduling Queues



- people standing in a line
- printer job queue
- CPU scheduling
- request handling in servers

3. Basic Operations of Queue

Operation	Description
Enqueue	Insert element at rear
Dequeue	Remove element from front
Front	Get front element
Rear	Get last element
isEmpty	Check if queue is empty
isFull	Check if queue is full

4. Queue Implementation using Array

Concept

Queue is implemented using:

- array
- two pointers: front and rear

Problem in Simple Array Queue

Even if elements are removed, unused space is not reused.

Example Issue

[10, 20, 30, _, _]

front = 2, rear = 2

Remaining space is wasted → leads to inefficiency

Program: Queue using Array

```
#include <iostream> using namespace std;

#define SIZE 5

class Queue {
    int arr[SIZE];
    int front, rear;
public:
    Queue() {
        front = rear = -1;
    }

    bool isFull() {
        return rear == SIZE - 1;
    }

    bool isEmpty() {
        return front == -1 || front > rear;
    }

    void enqueue(int x) {
        if (isFull()) {
            cout << "Queue Overflow\n";
            return;
        }

        if (front == -1)
            front = 0;

        arr[++rear] = x;
    }
}
```

```

void dequeue() {
    if (isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }

    cout << arr[front++] << " removed\n";
}

void display() {
    for (int i = front; i <= rear; i++)
        cout << arr[i] << " ";
    cout << endl;
}
};

```

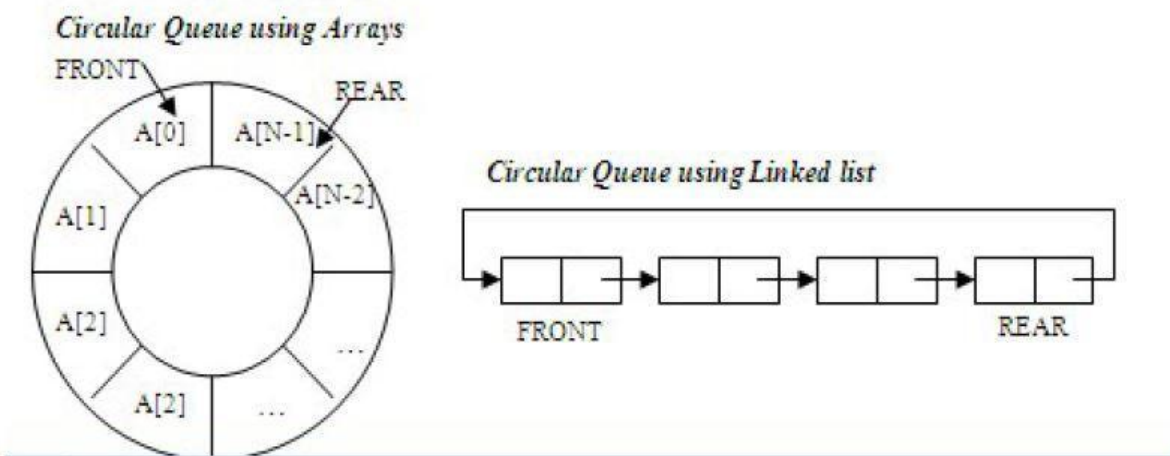
5. Optimized Queue: Circular Queue

Concept

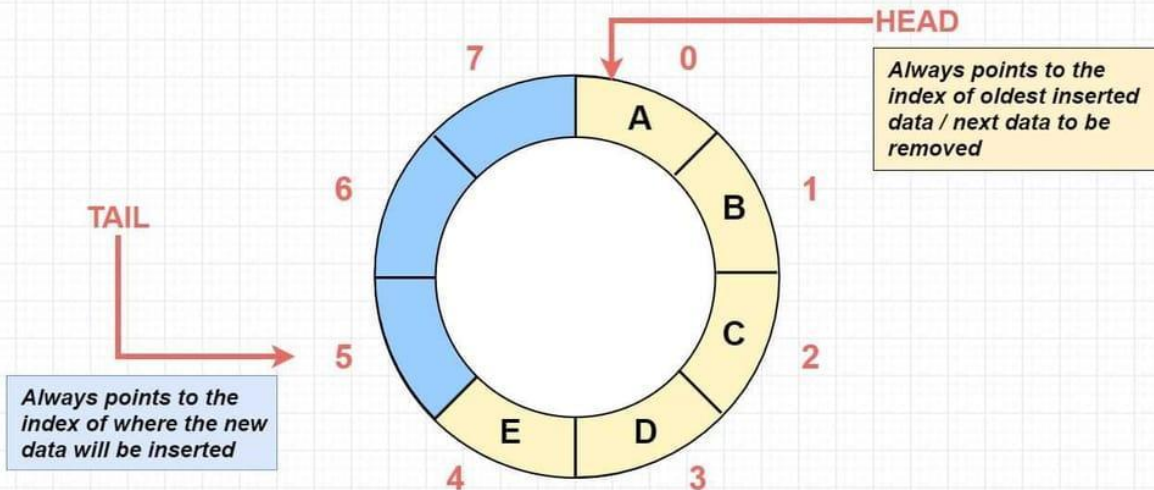
Circular queue solves memory wastage problem.

Instead of moving forward linearly, it **wraps around**.

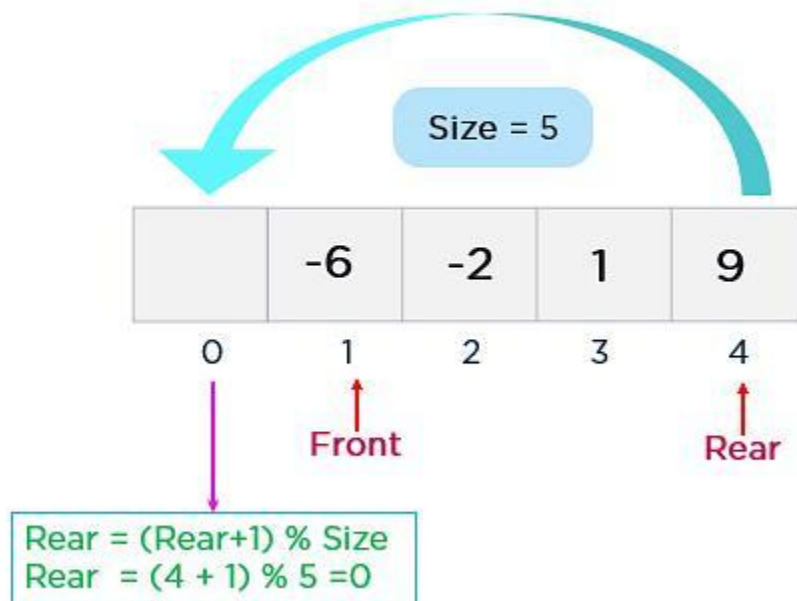
Visualization



Circular Queue (Ring Buffer)



Circular Incrementation



Key Formula

next = (rear + 1) % SIZE

Program: Circular Queue

```
1. #include <iostream> using namespace std;
#define SIZE 5

class CircularQueue {
    int arr[SIZE];
    int front, rear;
public:
    CircularQueue() {
        front = rear = -1;
    }

    bool isFull() {
        return (rear + 1) % SIZE == front;
    }

    bool isEmpty() {
        return front == -1;
    }

    void enqueue(int x) {
        if (isFull()) {
            cout << "Queue Full\n";
            return;
        }

        if (isEmpty())
            front = 0;

        rear = (rear + 1) % SIZE;
        arr[rear] = x;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue Empty\n";
            return;
        }

        cout << arr[front] << " removed\n";

        if (front == rear)
            front = rear = -1;
        else
            front = (front + 1) % SIZE;
    }
};
2.
```

6. Queue using Linked List

Concept

- dynamic memory allocation
- no overflow unless memory is full
- front and rear pointers used

Program: Queue using Linked List

```
1. #include <iostream> using namespace std;

class Node {
public:
    int data;
    Node* next;
};

class Queue {
    Node* front;
    Node* rear;
public:
    Queue() {
        front = rear = NULL;
    }

    void enqueue(int x) {
        Node* temp = new Node();
        temp->data = x;
        temp->next = NULL;

        if (rear == NULL) {
            front = rear = temp;
            return;
        }

        rear->next = temp;
        rear = temp;
    }

    void dequeue() {
        if (front == NULL) {
            cout << "Queue Empty\n";
            return;
        }

        Node* temp = front;
        cout << temp->data << " removed\n";
        front = front->next;

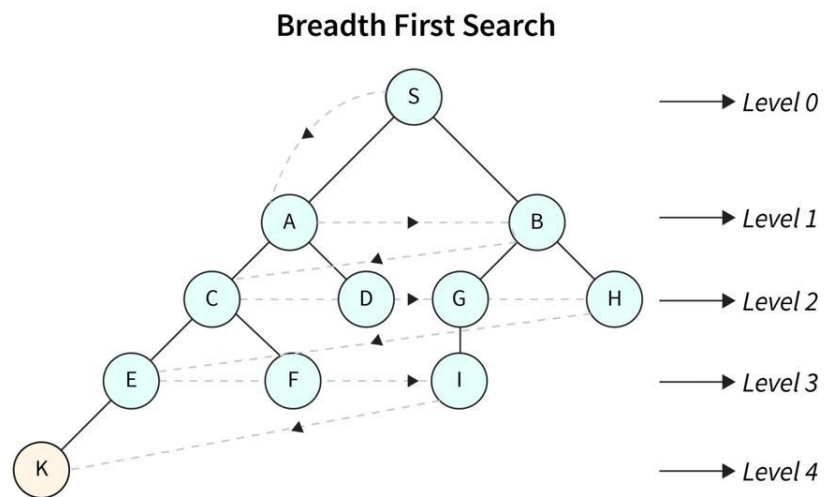
        if (front == NULL)
            rear = NULL;

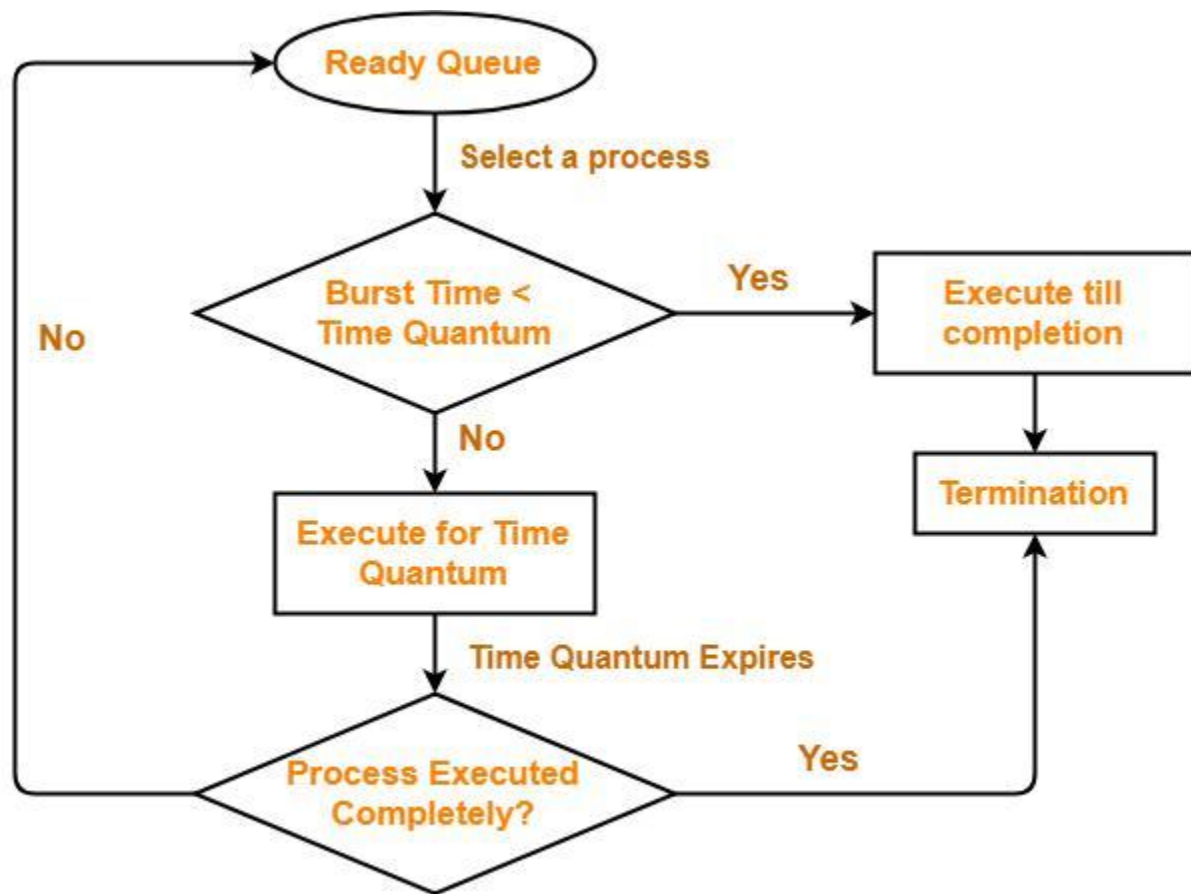
        delete temp;
    }
};
2.
```


7. Optimization Scenarios

Scenario	Solution
Memory wastage in array queue	Circular queue
Fixed size limitation	Linked list
High enqueue/dequeue	Circular queue
Dynamic size needed	Linked list
Efficient memory usage	Circular + dynamic

8. Applications of Queue





Round Robin Scheduling

- CPU scheduling (Round Robin)
- BFS (Breadth First Search)
- printer queue
- request handling systems
- buffering (keyboard, network)

9. Programming Tasks (Core)

Task 1: Reverse Queue

```
#include <queue>
```

```
#include <stack>
```

Use stack to reverse queue.

Task 2: Generate Binary Numbers

Input: n = 5

Output: 1, 10, 11, 100, 101

Task 3: First Non-Repeating Character in Stream

Task 4: BFS Traversal (Graph)

Task 5: Implement Queue using Two Stacks

10. LAB TASKS (To be Completed in Lab – 2 Hours)

Part A – Basic Implementation

1. Implement queue using array
2. Perform enqueue and dequeue

3. Handle overflow and underflow
4. Implement circular queue
5. Display queue elements

Part B – Intermediate Problems

1. Reverse a queue
2. Implement queue using linked list
3. Generate binary numbers using queue

11. HOME ASSIGNMENT (1 Week Submission)

Part C – Advanced Problems

Students must implement the following:

Q1: Implement Queue using Two Stacks

(Important conceptual problem)

Q2: Circular Linked List based Queue

- implement enqueue
- implement dequeue
- analyze complexity

Q3: First Non-Repeating Character in Stream

Example:

Input: a a b c

Output: a -1 b b

Q4: Sliding Window Maximum

Example:

Input: [1,3,-1,-3,5,3,6,7], k=3

Output: [3,3,5,5,6,7]

Q5: BFS Traversal of Graph

Q6: Design a Real-world System

Choose one:

- printer queue system
- call center queue
- task scheduling system

Implement using queue.

Q7: Priority Queue (Conceptual + Code)

12. Complexity Analysis

Operation	Array	Circular	Linked List
Enqueue	$O(1)$	$O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$	$O(1)$
Space	Fixed	Fixed	Dynamic

13. Viva Questions

1. What is FIFO?
2. Difference between stack and queue?
3. Why circular queue is needed?

4. What is overflow in queue?
5. What is difference between linear and circular queue?
6. Why linked list queue is dynamic?
7. What is BFS?
8. What is real-world use of queue?
9. What is priority queue?
10. Can queue be implemented using stack?

14. Submission Requirements

- handwritten algorithms
- dry run of problems
- complete C++ code
- output
- complexity analysis
- explanation of one real-world system